

Model Training Handoff Guide

Chandrayaan-2 optical imagery to LRO / SELENE registration

Perlin-augmented cross-sensor learning | Uniform correspondences |
Sub-pixel refinement

Purpose

Give this document directly to the teammate responsible for preparing the data, fine-tuning the matcher, running cloud training and validating the final model.

Recommended first milestone: OHRC to LRO registration at less than one source-image pixel RMSE.

EXECUTIVE BRIEF

1. What you are building

Build a model that receives a moving Chandrayaan-2 image and a fixed lunar reference image, predicts corresponding lunar points, removes outliers, estimates a geometric transformation and refines the surviving points to fractional-pixel coordinates.

Do not begin with all three sensors. The first deliverable must be a stable OHRC-to-LRO pipeline. Add TMC-2 and IIRS only after the complete OHRC pipeline passes the acceptance tests in Section 17.

System output

- Registered source image in the reference coordinate frame.
- CSV or GeoJSON containing source point, reference point, confidence and inlier status.
- Transformation matrix and transformation type.
- Match-line visualisation and spatial coverage map.
- RMSE, median error, PCK, inlier count, inlier ratio and runtime.

Recommended architecture

Stage	Recommended algorithm	Role
Preprocessing	Percentile normalisation + Sobel/CLAHE	Reduce sensor and illumination differences
Learned matching	Pretrained LoFTR-style matcher	Predict dense cross-image correspondences
Outlier rejection	MAGSAC++	Reject geometrically inconsistent matches
Uniformity	Grid quota + ANMS	Spread points across the valid image
Sub-pixel step	Local gradient NCC + soft-argmax	Return fractional coordinates
Warping	Homography / piecewise homography	Produce registered output

Definition of done

A model checkpoint is not the final solution. The final solution is the trained matcher plus MAGSAC++, uniform selection, sub-pixel refinement, image warping and a reproducible evaluation report.

EXECUTION SEQUENCE

2. Training plan at a glance

Phase	Input	Work	Exit condition
0. Baseline	20-50 real pairs	SIFT/AKAZE + MAGSAC++	Saved baseline metrics
1. Synthetic	LRO patches	Known homography + Perlin augmentation	Model learns scale, rotation and lighting invariance
2. Real fine-tune	OHRC-LRO overlaps	Metadata/manual/pseudo labels	Validation RMSE improves over baseline
3. Hard cases	Polar, shadowed, large-scale pairs	Curriculum of difficult augmentations	Stable performance across conditions
4. Sub-pixel	Original-resolution inlier windows	Covariance refiner	PCK@1 and PCK@0.5 reported
5. Extension	TMC-2 and IIRS	Sensor-specific inputs	No loss of OHRC performance

Non-negotiable rules

- Split by lunar region or orbit, never by random patches from the same parent image.
- Store the exact source-to-reference transformation direction for every synthetic sample.
- Save checkpoints every epoch to persistent storage.
- Evaluate on manually verified real correspondences, not only synthetic data.
- Keep the original high-bit-depth image; create normalised copies for learning.
- Run a 500-step benchmark before paying for a full cloud run.

Core mathematical objective

Model: $M_{\theta}(I_{\text{source}}, I_{\text{reference}}) \rightarrow \{(p_{\text{source}_i}, p_{\text{reference}_i}, \text{confidence}_i)\}$

Geometry: $p_{\text{reference}_i} \approx \text{project}(H * [x_{\text{source}_i}, y_{\text{source}_i}, 1]^T)$

Target: accurate, high-confidence, uniformly distributed matches with error < 1 source pixel

SETUP

3. Environment and project structure

Cloud recommendation

Use an RTX 5090 for the first OHRC model. Use an RTX PRO 5000 only when larger patches, larger batches or a joint multisensor model require more than 32 GB VRAM. Training happens on RunPod; the website only runs inference.

Required Python stack

```
python >= 3.10
torch, torchvision, kornia
opencv-python-headless, numpy, scipy, pandas
rasterio, GDAL, pds4_tools, spiceypy
albumations, noise, scikit-image
matplotlib, tensorboard, pyyaml, tqdm
```

Repository structure

```
lunarreg/
  configs/
    ohrc_lro.yaml
  data/
    raw/isro/ohrc/
    raw/nasa/lro/
    processed/source/
    processed/reference/
    labels/
    splits/
  src/
    preprocess.py
    build_pairs.py
    augment.py
    dataset.py
    model.py
    losses.py
    train.py
    infer.py
    geometry.py
    subpixel.py
    evaluate.py
  checkpoints/
  reports/
  requirements.txt
```

Suggested RunPod setup

- PyTorch/Jupyter template with a persistent 50-100 GB volume.
- Clone the repository and install requirements before uploading the full dataset.
- Keep raw PDS4 products outside the paid GPU instance where possible.
- Use TensorBoard logs and save best, last and per-epoch checkpoints.
- Stop the Pod immediately after downloading or preserving outputs.

DATASET ENGINEERING

4. Data acquisition and pairing

Data sources

Sensor	Use	Preferred pairing
Chandrayaan-2 OHRC	First moving/source sensor	LRO NAC for high resolution
Chandrayaan-2 TMC-2	Second-stage panchromatic sensor	LRO WAC, NAC or SELENE
Chandrayaan-2 IIRS	Later hyperspectral sensor	LRO WAC after band/PCA conversion
LRO NAC/WAC	Fixed geometric reference	Map-projected reference patches
SELENE	Alternative reference	Use when coverage/projection is suitable

Official starting portals

- ISRO PRADAN Chandrayaan-2: <https://pradan.issdc.gov.in/ch2/>
- ISRO Chandrayaan-2 MapBrowse: <https://chmapbrowse.issdc.gov.in/>
- LROC QuickMap: <https://quickmap.lroc.asu.edu/>

Pair-building procedure

- Download calibrated or map-projected products plus their XML/geometry labels.
- Extract the latitude-longitude footprint of each source product.
- Find the reference product covering the same lunar footprint.
- Reproject both products to one chosen lunar coordinate reference system.
- Resample to a manageable scale and identify the valid overlapping polygon.
- Create 512 x 512 overlapping patches with the same terrain content.
- Record parent product IDs so that geographic splitting remains possible.

Required manifest

```
pair_id,source_path,reference_path,source_sensor,parent_source,parent_reference,
min_lat,max_lat,min_lon,max_lon,split,label_type,transform_path,valid_mask_path
000001,processed/source/s001.tif,processed/reference/r001.tif,OHRC,
ohrc_product_17,lro_product_04,-70.2,-69.8,31.2,31.8,train,synthetic,
labels/000001_H.npy,labels/000001_mask.png
```

Never call two images a pair merely because both show craters. They must cover the same lunar coordinates.

MAKE THE SENSORS COMPARABLE

5. Preprocessing

Robust intensity normalisation

$$I_{\text{norm}}(x,y) = \text{clip}((I(x,y) - P2) / (P98 - P2 + \text{epsilon}), 0, 1)$$

P2 and P98 are the 2nd and 98th intensity percentiles of valid pixels.

Recommended operations

Operation	Reason	Instruction
Invalid-pixel masking	Prevents fake matches	Create a binary valid mask
Percentile clipping	Controls extreme values	Use valid pixels only
Gradient channel	Stabilises crater edges	Sobel magnitude or Laplacian
Mild CLAHE	Improves local contrast	Do not over-amplify sensor noise
Resolution alignment	Controls scale gap	Resample approximately, not perfectly
Patch filtering	Removes empty patches	Require valid terrain fraction

Sobel gradient

$$G_x = I * K_x \quad G_y = I * K_y$$

$$\text{Gradient magnitude } G = \sqrt{G_x^2 + G_y^2}$$

Use [normalised intensity, gradient magnitude] as a two-channel input, or use the gradient only for the final local correlation step.

IIRS conversion

Do not feed the entire hyperspectral cube into the first prototype. Use one high-SNR band or the first principal component. For PCA, centre the spectral matrix X , form $C = X^T X / (n-1)$, find the largest-eigenvalue eigenvector v_1 and calculate $PC1 = X v_1$.

Quality checks before training

- Overlay source and reference footprints in QGIS.
- Confirm coordinate direction and longitude convention.
- Save side-by-side contact sheets of random paired patches.
- Verify that synthetic labels remain inside the valid mask.
- Reject patches dominated by missing data or near-uniform darkness.

THE MOST IMPORTANT STAGE

6. Ground-truth generation

Training requires known correspondences. Use three label sources and store the label source for every pair.

Label type	How it is generated	Reliability
Synthetic	Known transform applied to one lunar patch	Highest geometric certainty
Metadata-derived	Map coordinates projected into both images	Depends on product geometry
Manual verified	Crater centres/rims selected in both images	Best real evaluation set
Pseudo-label	Classical/deep matcher + MAGSAC + review	Use only high-confidence inliers

Homography label

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix}$$

$$u = (h_{11} * x + h_{12} * y + h_{13}) / (h_{31} * x + h_{32} * y + 1)$$

$$v = (h_{21} * x + h_{22} * y + h_{23}) / (h_{31} * x + h_{32} * y + 1)$$

The saved H must always map SOURCE coordinates to REFERENCE coordinates.

Synthetic transformation curriculum

Difficulty	Rotation	Scale	Perspective	Translation
Easy	+/- 10 degrees	0.8-1.25	Very small	+/- 32 px
Medium	+/- 25 degrees	0.5-2.0	Small	+/- 96 px
Hard	+/- 45 degrees	0.25-4.0	Moderate	Overlap constrained

Do not start with extreme augmentation. First teach correct geometry on easy samples, then introduce scale, shadows, blur and viewpoint difficulty.

INNOVATION MODULE

7. Perlin augmentation

Perlin noise is used only to create smooth, realistic training perturbations. It is not the registration algorithm and should not be added to images during final inference.

Fractal Perlin field

$$\text{fade}(t) = 6*t^5 - 15*t^4 + 10*t^3$$

$$P(x,y) = \sum_o a^o * \text{noise}(2^o*x, 2^o*y) / \sum_o a^o$$

Recommended: 3-5 octaves, persistence $a = 0.5$.

Photometric augmentation

$$I_{\text{aug}}(x,y) = \text{clip}(I(x,y) * (1 + \alpha * P(x,y)) + \text{bias}, 0, 1)$$

Use α between 0.1 and 0.4.

Smooth displacement

$$dx(x,y) = \text{beta} * Px(x,y)$$

$$dy(x,y) = \text{beta} * Py(x,y)$$

$$x_{\text{new}} = x + dx(x,y) \quad y_{\text{new}} = y + dy(x,y)$$

Begin with $\text{beta} = 1-3$ px. Increase to 5 px only during hard-stage training.

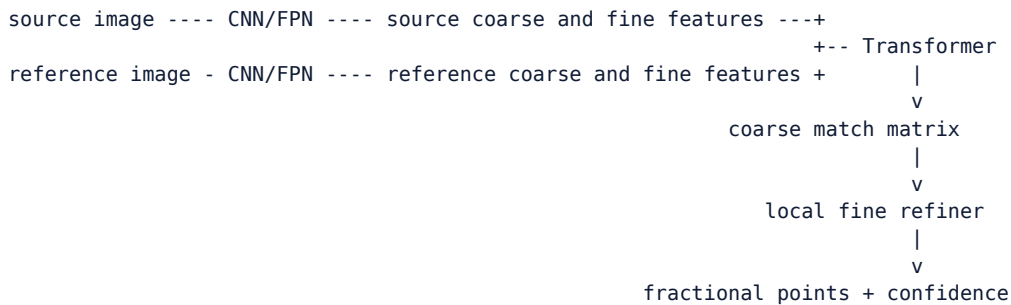
Required ablation experiment

Experiment	Purpose
A. SIFT + MAGSAC	Classical baseline
B. Pretrained matcher	Learning baseline
C. Real fine-tuned matcher	Benefit of cross-sensor training
D. Fine-tuned + normal augmentation	Benefit of standard augmentation
E. Fine-tuned + Perlin	Evidence for proposed innovation
F. Perlin + coverage + sub-pixel	Complete solution

LOFTR-STYLE ARCHITECTURE

8. Learned matching model

Forward path



Attention mathematics

$$Q = F \cdot W_q \quad K = F \cdot W_k \quad V = F \cdot W_v$$

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d}) * V$$

Use self-attention within each image and cross-attention between the images.

Descriptor similarity and dual-softmax

$$S_{ij} = \text{dot}(f_{\text{source}_i}, f_{\text{reference}_j}) / \text{temperature}$$

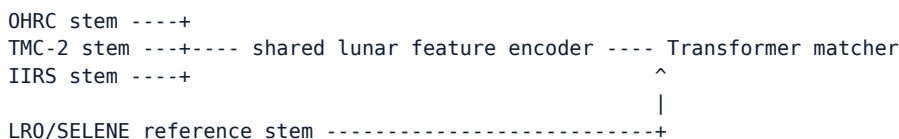
$$P_{ij} = \text{softmax_row}(S)_{ij} * \text{softmax_column}(S)_{ij}$$

Keep mutual nearest-neighbour pairs above the confidence threshold.

Implementation decision

Fine-tune a maintained pretrained implementation. Do not implement a Transformer matcher from zero for the first milestone. Replace or extend only the input channels, augmentation, losses and post-processing needed by this project.

Later multisensor architecture



OPTIMISATION OBJECTIVE

9. Training losses

Fine coordinate loss

$$\text{error}_i = \text{predicted_reference_point}_i - \text{ground_truth_reference_point}_i$$

$$\text{Huber}(\text{error}) = \begin{cases} 0.5 \cdot \text{error}^2 & \text{if } |\text{error}| \leq \delta \\ \delta * (|\text{error}| - 0.5 \cdot \delta) & \text{otherwise} \end{cases}$$

Cycle consistency

source point $\rightarrow$ predicted reference point $\rightarrow$ reconstructed source point

$$L_{\text{cycle}} = \text{mean}(\text{abs}(\text{source_point} - \text{reconstructed_source_point}))$$

Coverage loss

Divide the valid image into G cells. Let m_g be match-confidence mass in cell g .

$$q_g = m_g / (\sum_j m_j + \epsilon)$$

$$L_{\text{coverage}} = \sum_g (q_g - 1/G)^2$$

Apply only to valid cells that can genuinely contain correspondences.

Total objective

$$L_{\text{total}} = 1.0 \cdot L_{\text{coarse}} + 1.0 \cdot L_{\text{fine}} + 0.2 \cdot L_{\text{contrastive}} \\ + 0.1 \cdot L_{\text{cycle}} + 0.05 \cdot L_{\text{coverage}}$$

Initial optimiser configuration

```
optimizer: AdamW
learning_rate: 1.0e-4
weight_decay: 1.0e-4
betas: [0.9, 0.999]
scheduler: cosine
warmup_epochs: 2
gradient_clip_norm: 1.0
mixed_precision: true
```

RUN THE MODEL IN STAGES

10. Training procedure

Stage 0 - baseline

- Run SIFT and AKAZE on real OHRC-LRO pairs.
- Filter with ratio test and MAGSAC++.
- Record RMSE, inliers, ratio, coverage and runtime.

Stage 1 - synthetic warm-up

- Use LRO patches as base images.
- Generate source/reference pairs using known homographies.
- Add gradual illumination variation, blur and mild Perlin displacement.
- Train 10-15 epochs, beginning with frozen backbone layers if needed.

Stage 2 - real cross-sensor fine-tuning

- Mix synthetic pairs and genuine OHRC-LRO pairs.
- Use manually verified, metadata-derived or strictly filtered pseudo-labels.
- Train 15-25 epochs with a lower learning rate if validation becomes unstable.

Stage 3 - hard examples

- Increase sun-angle variation, shadow masks, scale differences and viewpoint distortion.
- Oversample polar and low-contrast cases without allowing them to dominate every batch.
- Train 5-10 epochs and stop when real validation no longer improves.

Stage 4 - sub-pixel refiner

- Freeze or lower the learning rate of the coarse matcher.
- Train local correlation windows using original-resolution patches.
- Optimise fractional coordinate error and uncertainty.

Starting configuration

```
image_size: 512
batch_size_5090: 8
batch_size_pro5000: 12
epochs_stage1: 15
epochs_stage2: 20
epochs_stage3: 8
checkpoint_every: 1
early_stopping_patience: 5
seed: 42
```

11. Dataset split and leakage prevention

A random patch split is invalid when neighbouring patches come from the same parent image. The model may memorise terrain and produce artificially strong results.

Split	Suggested proportion	Rule
Train	70%	Unique lunar regions and parent products
Validation	15%	No overlap with train footprints
Test	15%	Held-out regions with manual points

Required checks

- No parent product ID appears in multiple splits.
- No overlapping latitude-longitude footprint crosses split boundaries.
- Hard polar examples exist in validation and testing.
- Manual test points are never used as training pseudo-labels.
- Report synthetic and real test metrics separately.

Recommended dataset quantities

Milestone	Parent scenes	Generated pairs	Manual test pairs
Pipeline test	5-10	500-1,000	5-10
OHRC prototype	50-100	5,000-20,000	30-50
Strong submission	100+	20,000-50,000	50-100

12. Geometric verification

Normalised DLT

A homography needs at least four point pairs. Normalise point coordinates, build the linear system $Ah = 0$ and solve using singular value decomposition. Refit H using all accepted inliers and their confidence weights.

Symmetric transfer error

$$e_i = ||p_{\text{reference}_i} - \text{project}(H * p_{\text{source}_i})||^2 + ||p_{\text{source}_i} - \text{project}(\text{inv}(H) * p_{\text{reference}_i})||^2$$

MAGSAC++ sequence

- Discard matches below the neural confidence threshold.
- Run MAGSAC++ with the source-to-reference point arrays.
- Check minimum inlier count and inlier ratio.
- Check transform condition, scale and determinant for implausible solutions.
- Refit using all inliers with weighted normalised DLT.

Transformation choice

Model	Minimum points	Use
Similarity	2	Rotation, uniform scale and translation
Affine	3	Small viewpoint change without projective effects
Homography	4	Default for local lunar image patches
Piecewise homography	Many	Large regions with varying local geometry
Thin-plate spline	Many strong inliers	Only when residual deformation is spatially smooth

SIH REQUIREMENT

13. Uniform correspondence selection

RANSAC can retain many accurate points around one prominent crater. That does not satisfy the requirement for uniform distribution across the image.

Grid-quota algorithm

1. Divide the valid source area into an 8 x 8 or 10 x 10 grid.
2. Assign each geometric inlier to a cell.
3. Sort the cell's matches by confidence and local refinement quality.
4. Keep the top 1-5 matches per occupied cell.
5. Apply a minimum-distance or ANMS rule inside each cell.
6. Refit H and run one final reprojection-error filter.

Coverage metric

$$\text{coverage_percent} = 100 * \text{occupied_valid_cells} / \text{total_valid_cells}$$

Uniformity does not mean forcing a match into every cell. Textureless, invalid or non-overlapping cells must be excluded from the valid-cell denominator.

Acceptance guidance

- Aim for more than 60-70% valid-cell coverage on sufficiently textured overlaps.
- Retain at least 20-30 final inliers for a stable local homography when possible.
- Visualise occupied and empty cells in the evaluation report.

FINAL ACCURACY STAGE

14. Sub-pixel refinement

Sub-pixel accuracy is produced by local original-resolution refinement, not merely by using a larger GPU. Each MAGSAC inlier becomes the centre of a small local search.

Normalised cross-correlation

$$\text{NCC}(u,v) = \frac{\text{sum}((A - \text{mean}(A)) * (B_{uv} - \text{mean}(B_{uv})))}{\sqrt{\text{sum}((A - \text{mean}(A))^2) * \text{sum}((B_{uv} - \text{mean}(B_{uv}))^2)}}$$

Use gradient patches A and B rather than raw intensity when illumination differs.

Soft-argmax refinement

$$\begin{aligned} q_k &= \exp(C_k / \tau) / \sum_j \exp(C_j / \tau) \\ \delta_{\hat{}} &= \sum_k q_k * \text{offset}_k \\ p_{\text{reference_refined}} &= p_{\text{reference_coarse}} + \delta_{\hat{}} \end{aligned}$$

Starting settings

```
window_size: 31
search_radius: 4
input: gradient_magnitude
temperature: 0.05
max_allowed_shift: 4.0
reject_ambiguous_peaks: true
```

Quality rejection

- Reject flat correlation surfaces with no clear maximum.
- Reject points whose refined shift exceeds the local search radius.
- Reject refinements that increase symmetric reprojection error.
- Estimate uncertainty from correlation heatmap spread and save it with each point.

EVIDENCE FOR THE JURY

15. Metrics and evaluation report

Point RMSE

$$\text{RMSE} = \sqrt{\text{mean}(|\text{predicted_reference_point} - \text{ground_truth_reference_point}|^2)}$$

Additional metrics

Metric	Formula / definition	Why it matters
Median error	median(point error)	Less sensitive to large outliers
PCK@0.5	fraction with error < 0.5 px	Strict sub-pixel evidence
PCK@1.0	fraction with error < 1.0 px	Primary target
Inlier ratio	inliers / predicted matches	Robustness
Coverage	occupied valid cells / valid cells	Spatial uniformity
Success rate	successful pairs / total pairs	Operational reliability
Runtime	seconds per pair	Usability

Report results by condition

- Normal illumination versus strong sun-angle difference.
- Low, medium and large scale ratio.
- Equatorial versus polar terrain.
- OHRC, TMC-2 and IIRS separately.
- Before and after Perlin augmentation.
- Before and after sub-pixel refinement.

Honest target

Target less than 1 source-image pixel RMSE on the held-out manually verified test set. Do not claim guaranteed 0.2-pixel performance before the official evaluation data is available.

CLOUD HANDOFF

16. RunPod execution checklist

Before starting a paid Pod

- Unit-test the dataset loader on a CPU machine.
- Confirm one batch can complete forward and backward passes.
- Confirm paths, checkpoint directory and resume logic.
- Prepare a small 100-pair debug dataset.
- Verify the exact GPU hourly rate shown in the console.

First commands

```
git clone <repository-url> lunarreg
cd lunarreg
python -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
python -c "import torch; print(torch.cuda.get_device_name(0))"
python src/train.py --config configs/ohrc_lro.yaml --debug-batches 5
```

500-step benchmark

$\text{estimated_hours} = ((\text{number_of_pairs} / \text{batch_size}) * \text{epochs} * \text{seconds_per_step}) / 3600$
Add 15-25% for validation, checkpoint writing and data-loading variation.

Production run

```
python src/train.py \
+ --config configs/ohrc_lro.yaml \
+ --manifest data/splits/train.csv \
+ --val-manifest data/splits/val.csv \
+ --output checkpoints/ohrc_lro_run01 \
+ --resume-if-present
```

```
tensorboard --logdir checkpoints/ohrc_lro_run01/logs --bind_all
```

Files that must survive Pod shutdown

- best.ckpt, last.ckpt and final configuration YAML.
- Training and validation logs.
- Split manifests and dataset version/hash.
- Evaluation JSON/CSV and visual examples.
- Git commit hash used for the run.

BEFORE DECLARING THE MODEL TRAINED

17. Acceptance tests

Test	Pass condition
Loader	No missing paths; valid masks align with both images
Transform direction	Four-corner synthetic test maps source to reference correctly
Overfit test	Model can overfit 20-50 samples; otherwise training is broken
Leakage test	No parent or overlapping footprint appears across splits
Geometry	Estimated transforms have plausible scale, determinant and overlap
Real accuracy	Better than SIFT/AKAZE baseline on held-out real pairs
Sub-pixel	PCK@0.5 and PCK@1.0 are reported on manual points
Uniformity	Coverage computed using valid cells and visualised
Ablation	Perlin contribution measured, not merely described
Reproducibility	Checkpoint, config, seed, commit and manifests preserved

Submission target

The model is ready for integration when it consistently produces enough geometrically valid, uniformly distributed correspondences; achieves less than 1 source-pixel RMSE on the verified test set; and exports all metrics and artefacts automatically.

Do not hide failures

- Return a controlled failure when overlap is insufficient.
- Return a controlled failure when inliers or spatial coverage are too low.
- Never warp an image using an implausible transformation merely to produce an output.

TROUBLESHOOTING

18. Common failures and fixes

Problem	Likely cause	Fix
Loss does not fall	Wrong coordinate direction or labels	Visualise GT correspondences before training
Good synthetic, poor real	Domain gap	Increase real pairs and realistic photometric augmentation
Matches cluster	Prominent crater dominates	Grid quota, ANMS and coverage term
False shadow matches	Model learning intensity	Gradient channel and harder sun-angle examples
Out of memory	Large images/batch	AMP, smaller batch, gradient accumulation
Homography explodes	Outliers or poor point geometry	MAGSAC, plausibility checks, better spread
Sub-pixel worsens result	Ambiguous local correlation	Reject broad/multiple peaks and large shifts
Validation looks too good	Geographic leakage	Rebuild splits by parent footprint
IIRS matching fails	Spectral/radiometric gap	Use PCA1/high-SNR band and separate stem

Overfit sanity test

Before any long run, train on only 20-50 samples. The model should nearly memorise them. If it cannot, stop and fix the labels, loss, transforms or data loader. This small test prevents expensive cloud debugging.

DEVELOPER REFERENCE

19. Minimal implementation pseudocode

Training loop

```
for batch in train_loader:
    source, reference = batch['source'], batch['reference']
    gt_matches, valid_mask = batch['matches'], batch['valid_mask']

    source = perlin_and_geometric_augment(source, batch['transform'])

    with autocast(enabled=True):
        output = model(source, reference)
        loss = coarse_loss(output, gt_matches)
        loss += fine_loss(output, gt_matches)
        loss += 0.2 * contrastive_loss(output, gt_matches)
        loss += 0.1 * cycle_loss(output)
        loss += 0.05 * coverage_loss(output, valid_mask)

    optimizer.zero_grad(set_to_none=True)
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    clip_grad_norm_(model.parameters(), 1.0)
    scaler.step(optimizer)
    scaler.update()
```

Inference loop

```
pred = model(source, reference)
matches = confidence_filter(pred.matches)
H, inliers = magsac_homography(matches)
uniform = grid_quota_and_anms(inliers, valid_mask)
refined = local_gradient_ncc_subpixel(uniform, source_full, reference_full)
H_final = weighted_dlt(refined)
registered = inverse_warp(source_full, H_final)
metrics = evaluate(refined, ground_truth_if_available)
export_all(registered, refined, H_final, metrics)
```

Final teammate handoff

Start by producing the baseline, the pair manifest, the synthetic generator and a 500-step benchmark. Do not spend on a long RunPod session until those four items have been reviewed.

[PRINT THIS PAGE](#)

20. One-page execution checklist

- | |
|---|
| ☐ Calibrated OHRC and matching LRO products downloaded |
| ☐ Footprints verified and pair manifest created |
| ☐ Reprojection and patch extraction visually checked |
| ☐ Train/validation/test split made by lunar region |
| ☐ SIFT/AKAZE baseline metrics saved |
| ☐ Synthetic homography labels pass corner test |
| ☐ Perlin augmentation preserves crater geometry |
| ☐ Pretrained LoFTR-style checkpoint loads correctly |
| ☐ 20-50 sample overfit test passes |
| ☐ 500-step cloud benchmark completed |
| ☐ Stage 1 synthetic checkpoint saved |
| ☐ Stage 2 OHRC-LRO fine-tuned checkpoint saved |
| ☐ Stage 3 hard-condition checkpoint saved |
| ☐ MAGSAC++ and transform plausibility checks enabled |
| ☐ Grid coverage selection enabled |
| ☐ Original-resolution sub-pixel refinement enabled |
| ☐ RMSE, PCK, inlier ratio, coverage and runtime exported |
| ☐ Perlin ablation table generated |
| ☐ Best model, code commit, config and split manifests archived |

First target: OHRC -> LRO | Model: LoFTR-style | Geometry: MAGSAC++ homography | Final accuracy: local gradient NCC + soft-argmax